Scrum Poker – Leitfaden fürs Fachgespräch

Im Fachgespräch wird **jedes Gruppenmitglied einzeln** zu beliebigen Stellen im Code befragt. Wer nichts erklären kann, bekommt die Note 1.0, egal wie gut die App ist. Die App wurde grösstenteils mit KI erstellt (Dokumentation, Kapitel 15 «KI-Einsatz») und danach auf Wunsch der Gruppe **vereinfacht**: Sie benutzt nur Kursstoff, jede Datei beginnt mit einem deutschen Kommentar, und fast jede Funktion ist kurz genug, um sie in einer Minute zu erklären. Ihr müsst den Code jetzt so gut kennen, als hättet ihr ihn selbst geschrieben. Dieser Leitfaden zeigt, wie.

So arbeitet ihr mit diesem Leitfaden

1. **Teil A (Rundgang)** mit dem Code daneben lesen. Jede Datei öffnen und die genannten Zeilen ansehen. Zuerst den eigenen Bereich (siehe «Aufteilung für die Gruppe»), dann alles andere.
2. **Teil B (Musterantworten)** laut üben, **in eigenen Worten**, ohne abzulesen. Eine Person fragt, die andere zeigt die Stelle im Code und erklärt.
3. **Teil C (Fragenkatalog)** als Quiz verwenden.
4. **Teil E (Übungen)** wirklich durchführen: Code ändern, beobachten, zurücksetzen. Wer etwas selbst kaputt gemacht und repariert hat, kann es erklären.
5. **Teil D (Demo und Checkliste)** am Vortag einmal komplett durchspielen.

Verweise wie `PokerRoom.tsx:132` bedeuten Datei und Zeilennummer. Pfade sind relativ zum Projektordner (im Code-ZIP der Ordner `scrum-poker/`). Im Editor springt ihr mit `Ctrl+G` (VS Code: «Go to Line») direkt zur Zeile.

Lernplan: in vier Abenden bereit

Abend	Was	Ziel am Ende
1	App starten (`pnpm dev`), zwei Fenster, alles einmal durchklicken. Dann «Das Wichtigste in fünf Sätzen» (A.0) und den **eigenen Bereich** in Teil A lesen.	Ich kann «meine» Dateien Zeile für Zeile vorlesen und sagen, was passiert.
2	Die zwei **anderen Bereiche** in Teil A. Teil B.1–B.5 laut üben.	Ich kann jede Datei in einem Satz beschreiben und den Klick auf eine Karte vom Button bis zum Server verfolgen.
3	Teil B.6–B.12 und Teil C zu zweit als Quiz. Übungen E.1–E.8.	Ich beantworte die Fragen ohne Unterlagen und habe Code geändert und zurückgesetzt.
4	Restliche Übungen, Demo (Teil D) mit Stoppuhr, gegenseitig zu zufälligen Zeilen befragen.	Ich kann jede beliebige Stelle zeigen und erklären.

Kursthemen, auf die die Spalte «Kursthema» verweist:

Kürzel	Thema
HTML/CSS	semantisches HTML, Box-Modell, Flexbox, Grid, Responsive Design, Barrierefreiheit
TS	JavaScript/TypeScript: Types, Interfaces, Union Types, `unknown`, `typeof`-Prüfungen, `async`/`await`, Promises, `try`/`catch`
React	Komponenten, JSX, Props, Listen mit `key`, Events, bedingtes Rendern, `children`
State	`useState`, Lifting State Up, Single Source of Truth, abgeleitete Werte, kontrollierte Formulare
Effects	`useEffect`, Cleanup, Dependencies, StrictMode, `ignore`-Flag
Context	`createContext`, Provider, `useContext`, Context vs. Props
Routing	React Router: `Routes`, `Route`, `useParams`, `useNavigate`, `Link`

Kürzel	Thema
HTTP	REST, HTTP-Methoden, Statuscodes, JSON, `fetch`, Fehlerbehandlung, Polling mit `setInterval`
Express	Node.js, Express, Router, Middleware, Error-Handler
Test	Vitest, React Testing Library, supertest, Fake-`fetch`, Fake-Timer, Playwright
Tooling	pnpm-Workspace, Vite, Proxy, ESLint, `tsconfig`

TODO Gruppe: Falls der Kurs nummerierte Blöcke hat (z. B. «Block 4: Hooks»), die Nummern hier in der Tabelle ergänzen.

Aufteilung für die Gruppe

Der Code hat drei Bereiche. Jedes Mitglied nimmt sich **zuerst** einen Bereich vor, wird darin «Expertin» bzw. «Experte» und erklärt ihn danach den anderen. **Wichtig:** Das ist nur die Reihenfolge fürs Lernen. Im Fachgespräch muss **jede Person alles** erklären können, die Fragen kommen zu beliebigen Stellen.

Bereich	Dateien	Zentrale Fragen, die die Person zuerst beherrscht	Teil A
1. Server und gemeinsame Types	`shared/src/index.ts`, alle Dateien in `server/src/`, Server-Tests	REST-Routen, Methoden und Statuscodes; wie die Stimmen geheim bleiben; Heartbeat und Karteileichen; Durchschnitt bei T-Shirt-Grössen; warum der Server Eingaben prüft	A.2, A.3
2. Client-Basis und Home-Seite	`main.tsx`, `App.tsx`, `UserContext.tsx`, `api.ts`, `HomePage.tsx`, `NameForm.tsx`, `SessionListItem.tsx`, `ErrorBoundary.tsx`, `ErrorMessage.tsx`, `LoadingIndicator.tsx`, `NotFoundPage.tsx`, `global.css`, `HomePage.css`, Vite-Proxy	Routen und `useNavigate`; Context für den Namen und `localStorage`; Polling der Liste mit Cleanup; wie `api.ts` Fehler meldet; Grid und Box-Modell	A.4 (Einstieg bis Home-Seite)
3. Pokertisch	`SessionPage.tsx`, `SessionNameForm.tsx`, `SessionNotFound.tsx`, `PokerRoom.tsx`, `PokerTable.tsx`, `ParticipantSeat.tsx`, `CardDeck.tsx`, `DeckSwitcher.tsx`, `ShareLinkButton.tsx`, `RoundResult.tsx`, `SessionPage.css`, E2E-Tests	Beitreten/Verlassen im `useEffect`, StrictMode und `ignore`-Flag; Reload mit `sessionStorage`; Polling als Heartbeat; Klick auf eine Karte; Lifting State Up; Flexbox-Tisch	A.4 (Session-Seite bis Ende)

Vorschlag, wenn ihr mehr oder weniger als drei seid: Zu zweit nimmt eine Person Bereich 1 und 2 (Server und Home gehören über `api.ts` zusammen), die andere Bereich 3. Zu viert teilen sich zwei Personen den Pokertisch: `SessionPage.tsx` mit Beitreten/Verlassen und Reload für die eine, `PokerRoom.tsx` mit Polling, Aktionen und den Tisch-Komponenten für die andere.

So bringt ihr euch gegenseitig alles bei

1. Nach Abend 1 erklärt jede Person ihren Bereich den anderen in 15 Minuten, mit dem Code auf dem Bildschirm (nicht mit Folien).

2. Die Zuhörenden stellen danach je drei Fragen aus Teil C, die zu diesem Bereich passen.
3. Ab Abend 3 tauscht ihr: Jede Person erklärt einen Bereich, der **nicht** ihr eigener ist. Wer den Bereich vorbereitet hat, korrigiert.
4. Für den Datenfluss «Klick auf eine Karte» (B.4) braucht es alle drei Bereiche. Übt ihn so: Person 3 beginnt beim Button, Person 2 übernimmt bei `api.ts`, Person 1 erklärt Server und Store. Danach erzählt jede Person den ganzen Weg allein.

TODO Gruppe: Hier eintragen, wer welchen Bereich zuerst vorbereitet.

Teil A – Rundgang durch alle Dateien

A.0 Das Wichtigste in fünf Sätzen

1. Der **Server** (Express) hält alle Sessions in einer `Map` im Arbeitsspeicher und bietet dafür eine REST-API unter `/api/sessions` an.
2. Der **Client** (React) fragt den Server regelmässig nach dem aktuellen Stand (**Polling** mit `setInterval` im `useEffect`) und zeigt ihn an. Jede Klick-Aktion ist ein Request, dessen Antwort sofort angezeigt wird.
3. Der **Name** liegt in einem **Context** und im `localStorage`, die **eigene Teilnehmer-ID** im `sessionStorage` (damit ein Reload niemanden aus der Runde wirft).
4. **Beitreten** passiert im `useEffect` der Session-Seite, **Verlassen** in dessen Cleanup. Verschwindet jemand ohne Verlassen (Tab zu), entfernt ihn der Server nach 15–20 s, weil seine Polls fehlen (**Heartbeat**).
5. Types, Decks, Namensregel und Durchschnitt stehen nur einmal in **`shared/src/index.ts`**, Client und Server benutzen beide diese Datei.

A.1 Projektwurzel

Datei	Zweck	Wichtig zu wissen	Kursthema
`package.json`	Skripte für alle Pakete	`dev` startet Server und Client mit `concurrently` gleichzeitig. `test`, `build`, `typecheck` laufen mit `pnpm --recursive` in jedem Paket. `engines.node >= 22.12`, `packageManager: pnpm@10.34.6` (Corepack nimmt genau diese Version). `test:e2e` startet Playwright.	Tooling
`pnpm-workspace.yaml`	macht `client`, `server`, `shared` zu einem Workspace	Darum kann der Client `@scrum-poker/shared` mit `"workspace:*"` einbinden: «nimm den Ordner aus diesem Projekt», ohne ihn irgendwo zu veröffentlichen.	Tooling
`pnpm-lock.yaml`	genaue Versionen aller Abhängigkeiten	Gleiche Installation auf jedem Rechner. Nie von Hand ändern.	Tooling
`eslint.config.js`	ESLint für alle Pakete	`no-explicit-any` ist ein Fehler. `argsIgnorePattern: '^_'` erlaubt unbenutzte Parameter mit `_` (der Express-Error-Handler braucht vier Parameter). Für `client/` zusätzlich die React-Hooks-Regeln (prüfen u. a. die Dependency-Arrays).	Tooling

Datei	Zweck	Wichtig zu wissen	Kursthema
`.gitignore`	was nicht ins Git gehört	`node_modules/`, `dist/`, `*.tsbuildinfo`, Playwright-Ergebnisse, das Abgabe-ZIP	Tooling
`playwright.config.ts`	Konfiguration der E2E-Tests	startet Server und Client selbst auf eigenen Ports (3101/5273), ein Worker (die Tests teilen sich einen Server). Bewusst der Dev-Server: Nur dort läuft StrictMode.	Test
`e2e/scrum-poker.spec.ts`	End-to-End-Tests	zwei Browser-Kontexte = zwei Personen. Prüft den ganzen Ablauf, Tab schliessen, Fehlerzustände, kein horizontales Scrollen und eine saubere Browser-Konsole.	Test
`e2e/tsconfig.json`	Typprüfung der E2E-Tests	wird von `pnpm typecheck` mitgeprüft (`tsc -p e2e`)	TS
`README.md`	Kurzanleitung		
`docs/FACHGESPRAECH.md` / `.pdf` / `.html`	dieser Leitfaden		

Die Projektdokumentation (`DOKUMENTATION.pdf` / `.html`) wird neben dem Code-ZIP abgegeben, nicht darin. Im ZIP fehlen ausserdem die internen Ordner `scripts/` (Doku- und Abgabe-Scripts) und `deploy/` (Betrieb auf dem Server). Die App braucht sie nicht.

A.2 `shared/` – der gemeinsame Vertrag

`shared/src/index.ts` – Types und Funktionen, die Client und Server beide importieren. Kursthema: TS.

Zeilen	Was	Erklärung
7	`type DeckType = 'fibonacci' | 'tshirt'`	**Union Type aus Texten.** Nur diese zwei Werte sind erlaubt, ein Tippfehler wie `'tshrit'` ist ein Fehler beim Kompilieren.
9	`type CardValue = string`	Kartenwerte sind Texte (`'5'`, `'XL'`, `'?'`). Ob ein Wert gültig ist, prüft der Server zur Laufzeit (gehört er zum aktiven Deck?).
11-14	`interface Deck`	Ein Deck hat einen Namen für die Anzeige und eine Liste von Karten.
17-26	`DECKS`	Der Type `{ fibonacci: Deck; tshirt: Deck }` erzwingt, dass beide Decks da sind und jedes `label` und `cards` hat.
29	`DECK_TYPES`	Alle Deck-Namen als Liste. Der Client macht daraus die Buttons, der Server prüft damit.
33-39	`TSHIRT_VALUES`	Zahl pro T-Shirt-Grösse (XS=1 … XL=8) für den Durchschnitt. `Record<string, number>` heisst: ein Objekt mit Text-Schlüsseln und Zahl-Werten.
43	`MAX_NAME_LENGTH = 30`	Eine Konstante für Client **und** Server.
47-56	`validateName()`	Gibt die Fehlermeldung zurück oder `null`, wenn der Name gültig ist. Prüft den Namen ohne Leerzeichen am Rand (`trim()`). Benutzt von Home-Seite, Namensformular am Tisch und Server: eine Regel, gleiche Meldung überall.
60-67	`Participant`	Was der Client über eine Person sieht: `hasVoted` immer, `vote` nur nach dem Aufdecken oder bei der eigenen Stimme. `CardValue | null` zwingt den Client, den Fall «keine Stimme» zu behandeln.
69-74	`SessionState`	Der ganze Zustand eines Tisches. Antwort auf das Polling und auf alle Aktionen.

Zeilen	Was	Erklärung
77-82	`SessionSummary`	ein Eintrag der Liste auf der Home-Seite. `createdAt` ist Text, weil JSON keinen Datumstyp kennt.
87-118	Request- und Response-Types	Pro Route mit eigener Form ein Type, z. B. `CastVoteRequest { value: CardValue | null }`. Der Client baut damit den Body, der Server weiss, was er erwartet. `participantId?:` (Zeile 96) heisst: Das Feld darf fehlen.
116-118	`ErrorResponse { error: string }`	Form jeder Fehlerantwort
122-129	`RoundResult`	Ergebnis von `computeResult()`
132-145	`toNumber()`	T-Shirt → Zahl aus der Tabelle, sonst `Number(card)`. `'?'` und `'☕'` ergeben `NaN` → `null` (zählt nicht).
148-161	`closestTshirtSize()`	Geht alle Grössen durch und merkt sich die mit dem kleinsten Abstand. Das `<=` in Zeile 155 wählt bei Gleichstand die spätere, also grössere Grösse.
164-195	`computeResult()`	Erst Konsens (166-171), dann Summe und Anzahl der Zahlenkarten (174-182), dann Durchschnitt auf eine Nachkommastelle (189), bei T-Shirt die nächste Grösse (191-193). Gleiche Stimmen ergeben immer dasselbe Ergebnis, ohne Nebenwirkungen. Darum leicht zu testen.

`shared/src/index.test.ts` – elf Unit-Tests für `validateName()` und `computeResult()`. Kursthema: Test.

`shared/package.json`: `"exports": "./src/index.ts"` – das Paket zeigt direkt auf den TypeScript-Quelltext. Vite und `tsx` lesen TypeScript direkt, es braucht keinen Build-Schritt.

A.3 `server/` – Express-API

`server/src/index.ts` – Startpunkt des Servers. Kursthema: Express.

Zeile	Erklärung
9	`setInterval(removeInactiveParticipants, CLEANUP_INTERVAL_MS)` – alle 5 s Karteileichen entfernen (Kapitel 8 der Doku).
11-13	`app.listen(PORT, …)` startet den HTTP-Server auf Port 3001 und meldet «Scrum Poker API läuft auf …».

`server/src/app.ts` – die Express-App **ohne** `listen()`. So können die Tests die App mit supertest direkt aufrufen, ohne einen Port zu belegen. Kursthema: Express.

Zeile	Erklärung
10-12	Kommentar: Warum kein CORS nötig ist (Proxy in der Entwicklung, gleiche Adresse im Betrieb).
15	`app.use(express.json())` – **Middleware**: liest JSON-Bodies und legt sie in `req.body` ab.
17	`app.use('/api/sessions', sessionsRouter)` – alle Session-Routen hängen unter diesem Präfix.
20-31	Nur im Produktivbetrieb (`SERVE_CLIENT`): `express.static` liefert den gebauten Client aus. Andere `GET`-Adressen wie `/sessions/<id>` bekommen `index.html`, damit ein Reload auch ohne Vite funktioniert (React Router entscheidet dann). In der Entwicklung liefert Vite den Client, dort ist das aus.
34-36	Middleware **nach** allen Routen: Hat keine Route gepasst, kommt die Anfrage hier an → 404 als JSON statt einer HTML-Seite.

Zeile	Erklärung
39-48	**Error-Handler**, erkennbar an den **vier** Parametern `(err, req, res, next)`. `SyntaxError` kommt von `express.json()` bei kaputtem JSON → 400. Alles andere → 500; der Fehler wird nur in der Konsole ausgegeben, nie an den Client.

`server/src/config.ts` – alle Einstellungen an einem Ort: `PORT` (Zeile 7, Standard 3001, mit der Umgebungsvariable `PORT` änderbar), `CLEANUP_INTERVAL_MS = 5_000` (Zeile 10), `PARTICIPANT_TIMEOUT_MS = 15_000` (Zeile 14), dazu `CLIENT_DIST` und `SERVE_CLIENT` für den Produktivbetrieb (Zeilen 18-20). `5_000` ist dieselbe Zahl wie `5000`, der Unterstrich macht sie nur lesbarer.

`server/src/store.ts` – Speicher und Regeln, ohne Express. Kursthema: TS.

Zeilen	Erklärung
11-17	`StoredParticipant`: so speichert **der Server** eine Person, mit echter Stimme und `lastSeen` (Zeitpunkt der letzten Abfrage). Der Client bekommt diesen Type nie zu sehen.
19-25	`Session`: `createdAt` ist hier ein echtes `Date`, erst für die Antwort wird daraus Text.
28	`const sessions = new Map<string, Session>()` – **der ganze Speicher**: Schlüssel = Session-ID. Eine `Map` findet eine Session direkt über ihre ID.
34-54	`toSessionState()` macht aus der gespeicherten Session das, was der Client sehen darf. Zeile 38: Die Stimme wird nur mitgeschickt, wenn aufgedeckt ist **oder** es die eigene Stimme der fragenden Person ist. Sonst bleibt `vote` `null`. **Hier wird die Geheimhaltung gemacht.**
56-66	`listSessions()` – eine Zusammenfassung pro Session für die Home-Seite (Anzahl Personen statt Namen).
68-85	`createSession()` – 6 Zeichen einer zufälligen UUID als ID. Ist sie schon vergeben, wird neu gewürfelt (`while`). Neue Sessions starten mit Fibonacci und ohne Teilnehmer.
88-104	kleine Hilfsfunktionen: Session holen, löschen, alle löschen (nur für Tests), Person suchen. `undefined` heisst «gibt es nicht».
106-115	`addParticipant()` – neue Person mit `randomUUID()` als ID, ohne Stimme, `lastSeen = Date.now()`.
117-121	`removeParticipant()` – `filter` erzeugt eine neue Liste ohne die Person. Danach `revealIfEveryoneVoted`: War sie die Letzte ohne Stimme, wird aufgedeckt.
128-138	`removeInactiveParticipants()` – sucht alle, deren `lastSeen` länger als 15 s her ist, und entfernt sie. Leere Sessions bleiben stehen.
141-144	`castVote()` – Stimme setzen (`null` = zurückziehen), dann prüfen, ob alle gewählt haben.
147-157	`revealIfEveryoneVoted()` – leere Session: nichts tun. Hat jemand noch keine Stimme: nichts tun (`return`). Sonst aufdecken.
159-168	`revealCards()`, `resetRound()` – aufdecken bzw. alle Stimmen löschen und wieder verdecken.
170-177	`changeDeck()` – gleiches Deck: nichts tun. Anderes Deck: wechseln und neue Runde, weil alte Karten im neuen Deck nicht gelten.

`server/src/validation.ts` – prüft die Bodies. Kursthema: TS.

Ein Body kommt von aussen und kann alles enthalten. Darum ist er hier `unknown`. TypeScript lässt einen erst damit arbeiten, wenn man ihn geprüft hat.

Zeilen	Erklärung
9-29	`getJoinError()`. Zeile 10: Ist der Body überhaupt ein Objekt (`typeof body !== 'object'`), nicht `null` (in JavaScript ist `typeof null` auch `'object'`) und hat er ein Feld `name` (`'name' in body`)? Danach weiss TypeScript, dass `body.name` existiert. Zeile 17: Für den Namen gilt dieselbe Regel wie im Client (`validateName`). Zeilen 22-27: `participantId` darf fehlen, `null` oder Text sein.
32-44	`getVoteError()` – `null` ist erlaubt (zurückziehen). Sonst muss der Wert ein Text sein, der im aktuellen Deck vorkommt.
47-57	`getDeckError()` – das Deck muss einer der Namen aus `DECK_TYPES` sein.

Jede Funktion gibt die **Fehlermeldung** zurück oder `null`, wenn alles stimmt. Die Route macht daraus eine 400-Antwort.

`server/src/routes/sessions.ts` – alle Routen. Kursthema: Express, HTTP.

Jede Route liest sich gleich: Session holen → gibt es sie nicht, 404 → Body prüfen (400) → Store ändern → antworten.

Zeilen	Route	Erklärung
34-36	`GET /`	Liste aller Sessions, Status 200
39-43	`POST /`	Session erstellen → `201 { id }`
46-54	`DELETE /:sessionId`	404, wenn unbekannt, sonst löschen → `204` ohne Body (`res.status(204).end()`)
57-79	`GET /:sessionId`	Zustand eines Tisches. `req.query.participantId` (Zeilen 64-67) ist nur ein Text, wenn er in der Adresse steht. Zeilen 72-77: **Heartbeat**, `lastSeen` wird aktualisiert. Eine unbekannte ID ist kein Fehler.
82-111	`POST /:sessionId/participants`	Beitreten. Zeilen 88-92: Body prüfen. Zeile 94: `req.body as JoinSessionRequest` – ab hier ist der Body geprüft, `as` sagt TypeScript, welche Form er hat. Zeilen 98-106: Gibt es die mitgeschickte `participantId` noch, geht es mit ihr weiter (**200**, Reload). Sonst neue Person (**201**).
114-123	`DELETE /:sessionId/participants/:participantId`	Verlassen, **idempotent**: auch eine schon entfernte Person ergibt 204.
126-152	`PUT /:sessionId/participants/:participantId/vote`	Prüft der Reihe nach: Session (404), Person (404), schon aufgedeckt (409), Karte gültig (400). Antwort mit der eigenen Stimme (Zeile 151).
155-170	`PUT /:sessionId/deck`	Deck prüfen (400), wechseln, neuen Zustand schicken
173-181	`POST /:sessionId/reveal`	aufdecken
184-192	`POST /:sessionId/reset`	neue Runde

`req.params.sessionId` ist der Teil der Adresse an der Stelle von `:sessionId`.

Server-Tests (Kursthema: Test)

- **`server/src/app.test.ts`** – testet alle Routen mit **supertest**: `request(app).post(…)` schickt einen echten HTTP-Request an die App, ohne Port. Vor jedem Test leert `clearSessions()`

den Speicher. Der Heartbeat-Test ganz unten stellt mit `vi.useFakeTimers({ toFake: ['Date'] })` die Uhr vor, statt 15 s zu warten, und ruft dann `removeInactiveParticipants()` direkt auf.

- **`server/src/static.test.ts`** – testet den Produktivbetrieb mit einem kleinen künstlichen `client/dist`-Ordner.

A.4 `client/` – React-App

Einstieg und Konfiguration

`client/index.html` – die einzige HTML-Seite. Zeile 6: `viewport`, damit Handys die echte Breite verwenden. Zeile 10: `<div id="root">`, hier hängt React die App hinein. Zeile 11 lädt `main.tsx`. Kursthema: HTML/CSS.

`client/vite.config.ts` – Kursthema: Tooling.

Zeilen	Erklärung
9-10	Ports aus Umgebungsvariablen, sonst 5173 und 3001
16	`strictPort: true` – ist 5173 belegt, bricht Vite ab, statt still auf 5174 auszuweichen
19-21	**Proxy**: Jede Anfrage an `/api` geht an den Express-Server. Der Browser spricht nur mit `localhost:5173`, darum braucht es kein CORS.
23-27	Einstellungen für Vitest: `jsdom` spielt im Test den Browser, `setup.ts` läuft vor jedem Test-File.

`client/src/main.tsx` – Startpunkt. Kursthema: React, Context, Routing.

Zeilen	Erklärung
6-8	`global.css` wird **vor** `App` geladen. So kommen die Seiten-Styles danach und gewinnen bei gleich starken Regeln (z. B. der rote Button «Löschen»).
16	`createRoot(document.getElementById('root')!)` – das `!` sagt TypeScript: «dieses Element gibt es sicher» (es steht in `index.html`). Sonst wäre der Type `HTMLElement | null`.
17-24	Verschachtelung: `StrictMode` → `UserProvider` → `BrowserRouter` → `App`. Der Provider liegt aussen, damit jede Seite den Namen lesen kann.

`client/src/App.tsx` – Kursthema: Routing.

Zeilen	Erklärung
13	`ErrorBoundary` um alles: Fehler beim Rendern → Meldung statt weisser Seite.
14-18	Header mit `<Link to="/">`: wechselt die Seite ohne Neuladen.
20-25	`<Routes>` wählt **eine** `<Route>` passend zur Adresse. `:sessionId` (Zeile 22) ist ein Platzhalter, `*` (Zeile 24) passt auf alles andere.

`client/src/test/setup.ts` – läuft vor jedem Test-File: räumt nach jedem Test das gerenderte HTML ab und leert `localStorage` und `sessionStorage`, damit sich Tests nicht beeinflussen.

Context für den Namen

`client/src/UserContext.tsx` – Kursthema: Context, State.

Zeilen	Erklärung
7	Key im `localStorage`. Der Präfix `scrum-poker:` verhindert Kollisionen mit anderen Apps.
9-12	Was im Context steht: der Name und eine Funktion zum Ändern.
15	Kommentar für ESLint: Die Regel `react-refresh` möchte, dass eine Datei nur Komponenten exportiert. Context und Provider gehören aber zusammen, darum wird die Regel hier bewusst ausgeschaltet.

Zeilen	Erklärung
16	`createContext(…)` mit Startwert. Der Startwert gilt nur, wenn **kein** Provider darüber liegt; in der App kommt das nicht vor.
24	`useState(() => localStorage.getItem(…) ?? '')` – die Funktion läuft nur beim **ersten** Render, `localStorage` wird also nur einmal gelesen. `??` heisst: Ist links `null` (noch nichts gespeichert), nimm `''`.
27-30	`setName()` setzt den State **und** speichert sofort. Jeder Tastendruck ist gespeichert.
33	`<UserContext value={…}>` – seit React 19 ist der Context selbst der Provider. Alles in `children` kann den Wert mit `useContext(UserContext)` lesen.

Alle Anfragen an den Server

`client/src/api.ts` – eine Funktion pro API-Route. Komponenten rufen nie selbst `fetch()` auf. Kursthema: HTTP, TS.

Zeilen	Erklärung
21-32	`send()` – die einzige Stelle mit `fetch()`. Gibt es einen Body, wird er mit `JSON.stringify` zu Text und bekommt den Header `Content-Type: application/json`. **Wichtig:** `fetch()` wirft nur, wenn gar keine Antwort kommt (Server aus, kein Netz). Dann wird daraus «Server nicht erreichbar.» (Zeilen 27-31). Eine 404- oder 500-Antwort ist für `fetch()` kein Fehler.
35-50	`createError()` – macht aus einer Fehlerantwort ein `Error` mit Text. 502/503/504 kommen vom Vite-Proxy, wenn Express nicht läuft → auch «Server nicht erreichbar.». Sonst der Text aus `{ error }` vom Server. `as ErrorResponse` (Zeile 42): Wir vertrauen dem eigenen Server, dass die Antwort diese Form hat.
53-59	`listSessions()` – das Muster aller Funktionen: senden, `response.ok` prüfen (Status 200–299), sonst Fehler werfen, sonst JSON zurückgeben.
62-69	`createSession()` gibt nur die neue ID zurück.
72-81	`deleteSession()` – 404 zählt als Erfolg: jemand anderes hat die Session schon gelöscht, das Ziel ist erreicht.
86-95	`getSession()` – schickt die eigene `participantId` mit (Heartbeat). 404 → `null`: «diese Session gibt es nicht».
100-115	`joinSession()` – schickt Name und gespeicherte ID (nach Reload) mit. 404 → `null`.
118-161	`leaveSession()`, `castVote()`, `changeDeck()`, `revealCards()`, `resetRound()` – alle nach demselben Muster.
165-170	`getErrorMessage()` – im `catch` ist der Fehler vom Typ `unknown`. Mit `instanceof Error` prüfen wir, ob er eine `message` hat.

`client/src/api.test.ts` – ersetzt `fetch` durch einen Fake (`vi.stubGlobal`) und prüft Adresse, Body und Fehlermeldungen.

Home-Seite

`client/src/pages/HomePage.tsx` – Route `/`. Kursthema: State, Effects, HTTP.

Zeilen	Erklärung
16	`SESSION_LIST_POLL_INTERVAL_MS = 3000` – die Liste wird alle 3 s neu geladen.
23-25	Name aus dem Context. Ob er gültig ist, wird **bei jedem Render berechnet**, nicht gespeichert (abgeleiteter Wert, kann nie veralten).
28-31	Vier States. `useState<SessionSummary[] | null>(null)`: Der Type in spitzen Klammern sagt, was im State liegen darf. `null` heisst «noch keine Antwort».

Zeilen	Erklärung
34	`isLoading` – berechnet: Wir laden, solange weder Liste noch Fehler da sind.
37-64	**Polling-Effekt** (Muster, das auch `PokerRoom` benutzt): `let ignore = false` (39), `async function loadSessions()` mit `try` / `catch` (41-54), sofort einmal laden und dann alle 3 s (56-57), **Cleanup** `ignore = true; clearInterval(intervalId)` (60-63). Das leere Dependency-Array `[]` (64): Der Effekt läuft nur beim Öffnen der Seite.
48-53	Bei einem Fehler bleibt die alte Liste stehen, nur `listError` wird gesetzt. Der nächste Poll versucht es von selbst wieder.
66-77	`handleCreate()` – Ladezustand an, Session erstellen, dann `navigate('/sessions/' + sessionId)`. Bei Fehler Meldung und Button wieder frei.
81-90	`handleDeleted()` – entfernt die gelöschte Session sofort aus der Liste. `setSessions((currentSessions) => …)` mit einer Funktion bekommt den neusten Stand, auch wenn inzwischen ein Poll angekommen ist. `filter()` erzeugt eine **neue** Liste, der alte State wird nicht verändert.
94-98	Sortieren: zuerst kopieren (`[...sessions]`), denn `sort()` verändert das Array selbst, und State darf man nie direkt verändern.
110	`NameForm` bekommt Name, Fehlermeldung und `setName` als Props.
111-118	Button «Neue Session erstellen», gesperrt bei ungültigem Namen oder während des Erstellens.
125-143	Bedingtes Rendern mit `&&`: Fehler, Ladeanzeige, leere Liste, Hinweis, Liste. Zeile 140: `key={session.id}` – stabile ID statt Index.

`client/src/components/NameForm.tsx` – Kursthema: State, React.

Zeilen	Erklärung
8-13	Props: `name`, `error` (`null` = gültig), `onNameChange` (Callback nach oben).
16-19	`event.preventDefault()` – sonst würde der Browser beim Drücken von Enter die Seite neu laden.
30-31	**Kontrolliertes Eingabefeld**: `value` kommt von oben, jede Eingabe geht über `onNameChange` zurück in den Context.
32	`maxLength` – der Browser lässt nicht mehr als 30 Zeichen zu.
35-36	`aria-invalid` und `aria-describedby` sagen Screenreadern, dass das Feld ungültig ist und wo die Meldung steht (nur, wenn es eine gibt).
38-42	Meldung nur, wenn es einen Fehler gibt.

`client/src/components/SessionListItem.tsx` – Kursthema: State, React.

Zeilen	Erklärung
10-16	Props: die Session, ob Beitreten erlaubt ist, Callback nach dem Löschen.
21-22	Eigener State pro Eintrag: Nur dieser eine Eintrag zeigt «Wird gelöscht…».
24-28	Text «1 Person» / «x Personen» und die Uhrzeit (`toLocaleTimeString('de-CH', …)`).
32-46	`handleDelete()` – `window.confirm` fragt nach. Bei Erfolg `onDeleted(session.id)`: die Seite entfernt den Eintrag aus der Liste (Lifting State Up).
59-67	«Beitreten» navigiert nur auf die Session. Beigetreten wird erst auf der Session-Seite.
68-76	«Löschen» mit eigenem Ladetext.

`client/src/components/ErrorBoundary.tsx` – die **einzige Klasse** der App, weil React Error Boundaries nur als Klasse anbietet. Zeilen 20-22: `getDerivedStateFromError` setzt `hasError: true`, wenn ein Kind beim Rendern einen Fehler wirft. Zeilen 25-36: dann eine Meldung statt der Kinder.

Zeile 31: `<Link reloadDocument>` lädt die ganze App neu, damit auch der Fehlerzustand weg ist. Fängt **keine** Fehler in Event-Handlern oder `fetch` (die fangen wir mit `try` / `catch`). Kursthema: React.

`client/src/components/ErrorMessage.tsx` – rote Meldung mit `role="alert"` (Screenreader lesen sie sofort vor). Mit `onRetry` erscheint der Button «Erneut versuchen» (Zeilen 14-18), ohne nicht. Das `?` in `onRetry?:` heisst: Die Prop ist freiwillig.

`client/src/components/LoadingIndicator.tsx` – drehender Kreis und Text mit `role="status"`. Der Kreis hat `aria-hidden="true"`, weil er nur Dekoration ist.

`client/src/pages/NotFoundPage.tsx` – für alle unbekannten Adressen, mit `<Link to="/">` zurück.

`client/src/pages/HomePage.test.tsx` – ersetzt `fetch` durch einen Fake-Server und prüft Name, Erstellen, Liste, Fehler, Polling (mit Fake-Timern: «nach 3 s kommt der zweite Request, nach dem Verlassen der Seite keiner mehr») und Löschen. **`client/src/App.test.tsx`** prüft die Routen. **`client/src/UserContext.test.tsx`** prüft Lesen und Schreiben im `localStorage`.

Session-Seite (Beitreten, Verlassen, Reload)

`client/src/pages/SessionPage.tsx` – Route `/sessions/:sessionId`. Kursthema: Effects, Routing, State.

Zeilen	Erklärung
17	`const { sessionId = '' } = useParams()` – liest `:sessionId` aus der Adresse. `useParams()` liefert `string | undefined`, `= ''` macht daraus sicher einen Text.
18	Name aus dem Context.
22-23	`hasName` und `trimmedName` werden berechnet.
25-29	Vier States. `joinAttempt` ist ein Zähler: Hochzählen startet den Effekt neu («Erneut versuchen», «Wieder beitreten»).
34-93	**Join-Effekt.** Beitreten beim Öffnen, Verlassen im Cleanup.
35-37	Ohne gültigen Namen nichts tun. Dann zeigt die Seite das Namensformular.
43-44	Gespeicherte ID aus `sessionStorage` lesen (gibt es nur nach einem Reload in diesem Tab).
49-50	`ignore` (Muster aus der React-Doku) und `joinedId` (wer ist beigetreten, damit das Cleanup weiss, wen es abmelden muss).
52-76	`async function join()` : `joinSession(...)` (54). Ist der Effekt schon aufgeräumt (`ignore`, 55-63), wird ein **neu** angelegter Teilnehmer gleich wieder abgemeldet. `null` → «nicht gefunden» (64-67). Sonst ID merken, in `sessionStorage` schreiben und anzeigen (68-70). Fehler → Meldung (71-75).
78	`join()` wird aufgerufen. Ein Effekt selbst darf nicht `async` sein, darum die innere Funktion.
81-92	**Cleanup**: `ignore = true`; war man beigetreten, Leave-Request schicken (Fehler egal, `.catch(() => {})`) und die gespeicherte ID löschen; State zurücksetzen.
93	Dependencies: Ändert sich Session, Name oder `joinAttempt`, wird zuerst aufgeräumt und dann neu beigetreten.
95-97	`joinAgain()` zählt `joinAttempt` hoch.
99-114	`renderContent()` entscheidet mit `if` und frühem `return`, was angezeigt wird: Namensformular, «nicht gefunden», Fehler, Laden oder der Tisch.
113	`key={participantId}`: neue ID → React baut ein frisches `PokerRoom` mit leerem State.

`client/src/components/SessionNameForm.tsx` – fragt nach dem Namen, wenn jemand einen geteilten Link ohne gespeicherten Namen öffnet. `draft` (Zeile 10) ist ein eigener State: Erst beim Absenden (Zeilen 14-20) kommt der Name in den Context, sonst würde schon beim ersten

Buchstaben beigetreten. Zeile 12: dieselbe Prüfung `validateName` wie auf der Home-Seite. Danach sieht `SessionPage` einen gültigen Namen, und ihr Effekt tritt von selbst bei.

`client/src/components/SessionNotFound.tsx` – Meldung «Diese Session existiert nicht (mehr)» mit Link zurück. Benutzt von `SessionPage` (Beitreten schlägt fehl) **und** `PokerRoom` (Session wird gelöscht, während man am Tisch sitzt): derselbe Fehlerzustand.

Pokertisch

`client/src/components/PokerRoom.tsx` – der Tisch eines beigetretenen Teilnehmers. Hält den ganzen Zustand des Tisches. Kursthema: State, Effects, HTTP.

Zeilen	Erklärung
19	`POLL_INTERVAL_MS = 1500` – muss deutlich unter dem Server-Timeout von 15 s liegen, weil jeder Poll als Heartbeat zählt.
21-25	Props: Session-ID, eigene ID, Callback für «Wieder beitreten».
32-37	State: `session` (Zustand vom Server, Single Source of Truth), `pollError`, `notFound`, `isBusy` (eine Aktion läuft), `actionError`.
40-76	**Polling-Effekt**, gleiches Muster wie auf der Home-Seite.
49-55	Session gelöscht (`null`): `clearInterval` stoppt das Polling, damit nicht alle 1.5 s ein unnötiger Request mit 404 rausgeht. Dann `notFound`.
58-63	Fehler: der letzte Stand bleibt sichtbar, `pollError` zeigt die Meldung.
67-68	Erst den Timer anlegen, dann sofort laden. So kennt `loadSession` die `intervalId` schon, wenn die erste Antwort kommt (sie kommt erst nach dem `await`).
72-75	Cleanup: `ignore = true` und `clearInterval`. Ohne das liefe das Polling nach dem Verlassen weiter.
76	Dependencies `[sessionId, participantId]`.
81-91	`runAction(request)` – für alle vier Aktionen gleich: `isBusy` an, alten Fehler weg, auf die Antwort warten, sie mit `setSession` sofort anzeigen; bei Fehler Meldung; `isBusy` aus. Der Request läuft schon, wenn `runAction` ihn bekommt, z. B. `runAction(revealCards(sessionId))`.
93-102	Frühe Returns: nicht gefunden, noch nichts geladen (Laden oder Fehler).
104-116	`me` – die eigene Person in der Liste suchen. Fehlt sie, hat der Server uns entfernt → «Du bist nicht mehr am Tisch» mit «Wieder beitreten».
118-120	Abgeleitete Werte: eigene Stimme, Anzahl Stimmen. Nicht im State, sondern bei jedem Render berechnet.
123-130	`roundStatus` – die Tischmitte: vor dem Aufdecken «x von y haben gewählt», danach `RoundResult`. Ein `if` statt eines verschachtelten Ausdrucks im JSX.
132-139	`handleCardClick()` – Klick auf die schon gewählte Karte → `null` (zurückziehen), sonst die Karte.
141-151	Die anderen Aktionen, jede eine Zeile.
155	`aria-busy` sagt Screenreadern, dass gerade etwas läuft.
160-164	«Verlassen» navigiert **nur**. Das Leave macht das Cleanup in `SessionPage`.
171-185	`PokerTable` bekommt die Tischmitte als `children`: Stand oder Ergebnis, «Wird gesendet …» während einer Aktion, die Buttons «Karten aufdecken» (gesperrt, solange niemand gewählt hat) und «Neue Runde».
187-192	`CardDeck` bekommt Deck, eigene Stimme und `handleCardClick`. Gesperrt, wenn aufgedeckt ist oder eine Aktion läuft.

`client/src/components/PokerTable.tsx` – die Plätze. Zeilen 16-18: Die erste Hälfte sitzt oben, die zweite unten (`Math.ceil` rundet auf, `slice` kopiert einen Teil der Liste). Zeilen 23-30 und 34-41: ein `ParticipantSeat` pro Person mit `key={participant.id}`. Zeile 32: `{children}` – was `PokerRoom` mitgibt, landet in der Tischmitte. Kursthema: React.

`client/src/components/ParticipantSeat.tsx` – ein Platz. Kursthema: React, HTML/CSS.

Zeilen	Erklärung
12-23	`describeStatus()` – Text für Screenreader («wählt noch», «hat gewählt», «Karte 5»).
27-35	Welche Karte gezeigt wird: leer, verdeckt (`--covered`) oder aufgedeckt mit Wert (`--revealed`). Drei CSS-Klassen, keine Animation.
38	Der eigene Platz bekommt zusätzlich `seat--own` (Rahmen in Farbe).
39	`aria-hidden="true"` – die Karte ist nur Dekoration, der Text steht in Zeile 46.
46	`visually-hidden` – unsichtbar, aber Screenreader lesen es vor.

`client/src/components/CardDeck.tsx` – die eigenen Karten. Kein eigener State. Zeile 20: `DECKS[deck].cards.map(...)` – eine Karte pro Wert des aktiven Decks. Zeile 24: `key={value}` (die Werte sind im Deck eindeutig). Zeilen 26-27: Klasse und `aria-pressed` für die gewählte Karte. Zeile 29: `onClick={() => onCardClick(value)}` – die Arrow-Function gibt den Wert dieser Karte mit. Kursthema: React, State.

`client/src/components/DeckSwitcher.tsx` – zwei Buttons, einer pro Deck aus `DECK_TYPES`. Zeilen 12-17: Klick auf das aktive Deck tut nichts (sonst würde die Runde unnötig zurückgesetzt). Zeilen 20-21: `fieldset` mit `legend` gruppiert die Buttons. Zeile 28: `aria-pressed` zeigt das aktive Deck.

`client/src/components/ShareLinkButton.tsx` – Kursthema: Effects.

Zeilen	Erklärung
10-16	Zweiter Effekt mit Cleanup: Nach «Kopiert!» startet ein Timer, der nach 2 s zurückschaltet. Das Cleanup `clearTimeout` verhindert, dass der Timer feuert, wenn der Button vorher verschwindet.
20	`window.location.href` wird nur **gelesen**, nie zum Navigieren benutzt.
23	`navigator.clipboard.writeText` – gibt es nur unter `https` oder auf `localhost`.
27	Notlösung: Link in einem Dialog anzeigen.
34	`aria-live="polite"` – Screenreader lesen «Kopiert!» vor.

`client/src/components/RoundResult.tsx` – Ergebnis nach dem Aufdecken. Kursthema: React, State.

Zeilen	Erklärung
13-23	Eine `for … of` -Schleife sammelt die Stimmen. Wer nicht gewählt hat, wird übersprungen (`continue`). `?` und `☕` werden gezählt, um sie unten zu erwähnen.
24	`computeResult()` aus `shared`. Das Ergebnis ist kein State, sondern wird bei jedem Render neu berechnet.
33	Konsens hervorgehoben (gelber Kasten, CSS-Klasse `round-result__consensus`).
34-43	`<dl>`: Beschreibungsliste für «Durchschnitt» und «Stimmen». `?? '–'` zeigt einen Strich, wenn es keinen Durchschnitt gibt.
44-49	T-Shirt: Erklärung, wie die Grösse berechnet wurde.

`client/src/pages/SessionPage.test.tsx` – Kursthema: Test. Zeilen 30-83: ein kleiner **Fake-Server**, der `fetch` ersetzt und wie die echte API antwortet. `renderSessionPage()` rendert die Seite

in einem `MemoryRouter` (Router ohne echten Browser), auf Wunsch in `<StrictMode>`. Die Tests prüfen Beitreten, Verlassen, Reload, StrictMode, Karten, Ergebnis, Fehler und «Kopiert!».

CSS

Datei	Inhalt	Kursthema
`client/src/styles/global.css`	Variablen für Farben und Abstände (`--space-md` …), `box-sizing: border-box` für alle Elemente, Schrift, Header, Buttons (`.button` , `.button--secondary`), Fehlermeldung, Ladeanzeige, `.visually-hidden` , `:focus-visible` , `prefers-reduced-motion` . **Mobile First**: Grundregeln für das Handy, `@media (min-width: 768px)` für grössere Bildschirme.	HTML/CSS
`client/src/pages/HomePage.css`	**Grid**: eine Spalte auf dem Handy, ab 768 px zwei Spalten 2 : 3. Namensfeld, Liste (Flexbox), roter Button «Löschen» (`.button--danger`).	HTML/CSS
`client/src/pages/SessionPage.css`	alle Tisch-Styles in einer Datei: Namensformular, Werkzeugleiste, Deck-Umschalter, **Flexbox**-Tisch (Handy: Plätze als Liste, Tisch darunter mit `order: 1` ; ab 768 px: Plätze oben und unten, ovaler Tisch dazwischen), Plätze, drei Kartenarten, Ergebnis, Handkarten mit `flex-wrap` .	HTML/CSS

Teil B – Musterantworten auf die typischen Fragen

B.1 «Zeigen Sie Komponente X. Welche Props hat sie, wer rendert sie?»

Vorgehen: Datei öffnen → Kommentar oben vorlesen (wozu, wer benutzt sie) → `interface …Props` zeigen → sagen, was jede Prop bedeutet → mit `Ctrl+Shift+F` nach `<Komponentenname` suchen und die Stelle zeigen, die sie rendert.

> **Beispiel `CardDeck` :** « `CardDeck` zeigt die eigenen Karten. Sie bekommt vier Props: `deck` , welches Deck aktiv ist; `selected` , die eigene Stimme oder `null` ; `disabled` , ob die Karten gesperrt sind; und `onCardClick` , eine Funktion, die sie beim Klick mit dem Kartenwert aufruft. Gerendert wird sie von `PokerRoom` in Zeile 187. Sie hat keinen eigenen State: Welche Karte gewählt ist, kommt vom Server über `PokerRoom` .»

> **Beispiel `PokerRoom` :** « `PokerRoom` ist der Tisch einer Person, die beigetreten ist. Props: `sessionId` , `participantId` und `onRejoin` . Gerendert wird er von `SessionPage` erst, wenn das Beitreten geklappt hat, mit `key={participantId}` . Er hält den Zustand des Tisches, pollt ihn alle 1.5 s und gibt alles per Props an `DeckSwitcher` , `PokerTable` und `CardDeck` weiter.»

Die Tabelle aller Komponenten steht in der Dokumentation, Kapitel 3.2.

B.2 «Warum liegt dieser State hier?»

Grundregel: **State liegt in der untersten Komponente, die ihn ändern muss, und gerade so weit oben, dass alle, die ihn lesen, darunter liegen.**

State	Ort	Satz zum Merken
Name	Context über dem Router	Home **und** Tisch brauchen ihn, und er soll den Seitenwechsel überleben.
Session-Zustand	`PokerRoom`	Alle Teile des Tisches brauchen ihn; `PokerRoom` ist der gemeinsame Elternteil (Lifting State Up).

State	Ort	Satz zum Merken
`isDeleting`	`SessionListItem`	Betrifft nur diesen einen Eintrag.
`draft`	`SessionNameForm`	Ein Entwurf, erst beim Absenden zählt er.
eigene Stimme	**kein** eigener State	Steht schon im Server-Zustand (`me.vote`). Doppelt gespeichert könnte sie veralten (Single Source of Truth).
Ergebnis, Gültigkeit des Namens	**kein** State	Wird bei jedem Render berechnet.

B.3 «Welche Routen gibt es, wie kommt die Session-ID in die Komponente?»

> «Drei Routen in `App.tsx`, Zeilen 20-25: `/` für die Home-Seite, `/sessions/:sessionId` für den Tisch und `*` für alles andere. `:sessionId` ist ein Platzhalter. `SessionPage` liest ihn in Zeile 17 mit `useParams()` aus und gibt ihn als Prop an `PokerRoom` weiter. Weil die ID in der Adresse steht, kann man den Link teilen und die Seite neu laden. Programmatisch navigieren wir mit `useNavigate()`: nach dem Erstellen einer Session, beim Klick auf «Beitreten» und bei «Verlassen». Für Links im Header und auf den Fehlerseiten nehmen wir `<Link>`, nie `<a href>`, damit die Seite nicht neu lädt.»

B.4 «Verfolgen Sie einen Klick auf eine Karte.»

Mit dem Code daneben, in dieser Reihenfolge zeigen (Details: Dokumentation, Kapitel 6):

1. `CardDeck.tsx:29` – `onClick={() => onCardClick(value)}` meldet den Wert nach oben.
2. `PokerRoom.tsx:132-139` – `handleCardClick`: schon gewählt → `null`, sonst der Wert.
3. `PokerRoom.tsx:81-91` – `runAction`: `isBusy` an, alles gesperrt.
4. `api.ts:126-133` – `castVote`: `PUT /api/sessions/:id/participants/:pid/vote` mit `{ "value": "5" }`.
5. `vite.config.ts:19-21` – der Proxy leitet an Port 3001 weiter.
6. `sessions.ts:126-152` – Route prüft: Session (404), Person (404), aufgedeckt (409), Karte (400).
7. `store.ts:141-157` – Stimme speichern, alle gewählt? → aufdecken.
8. `sessions.ts:151` + `store.ts:38` – Antwort mit der eigenen Stimme, die anderen bleiben `null`.
9. `PokerRoom.tsx:86` – `setSession(newSession)` → React rendert neu → Karte hervorgehoben.
10. Die anderen sehen die verdeckte Karte bei ihrem nächsten Poll (≤ 1.5 s).

B.5 «Welche API-Routen gibt es, welche Methode, warum?»

> «Alle unter `/api/sessions` in `server/src/routes/sessions.ts`. `GET` zum Lesen: die Liste und der Zustand eines Tisches. `POST` erstellt etwas Neues, dessen ID der Server vergibt: eine Session oder einen Teilnehmer, Antwort 201. `PUT` setzt einen Wert vollständig: die eigene Stimme und das Deck; zweimal derselbe Request ergibt dasselbe (idempotent). `DELETE` entfernt: Session oder Teilnehmer, Antwort 204 ohne Body. Aufdecken und Neue Runde sind Aktionen, darum `POST`. Fehler kommen immer als `{ error: '…' }`: 400 bei ungültiger Eingabe, 404 wenn etwas nicht existiert, 409 wenn man nach dem Aufdecken noch abstimmen will.»

Die ganze Tabelle steht in der Dokumentation, Kapitel 5.

B.6 «Was passiert, wenn der Server beim Polling nicht antwortet?»

> «`fetch` wirft, weil keine Antwort kommt. `send()` in `api.ts`, Zeilen 27-31, macht daraus den Fehler ‹Server nicht erreichbar.›. Über den Vite-Proxy kommt stattdessen eine 502, die `createError()` in denselben Text übersetzt. Im Polling-Effekt von `PokerRoom` landet der Fehler im `catch`, Zeilen 58-63: `pollError` wird gesetzt, `session` aber **nicht** gelöscht. Darum bleibt der Tisch mit dem letzten Stand sichtbar, darüber steht ‹… Neuer Versuch läuft …›. Das Intervall läuft weiter; sobald der Server wieder antwortet, setzt der nächste Poll `pollError` auf `null`. Startet der Server neu, sind die Daten weg, die nächste Antwort ist 404, also `null`, und wir zeigen ‹Diese Session existiert nicht (mehr)›. Die Home-Seite macht es genauso.»

B.7 «Warum ist dieser Typ so definiert? Was ginge ohne ihn schief?»

- **`DeckType = 'fibonacci' | 'tshirt'`:** Mit `string` könnte man `'tshrit'` schreiben, und es fiele erst zur Laufzeit auf. Der Union Type lässt nur die zwei Werte zu.
- **`vote: CardValue | null`:** `null` heisst «keine Stimme» bzw. «verdeckt». TypeScript zwingt jeden, der `vote` benutzt, an diesen Fall zu denken (z. B. `participant.vote ?? ''` in `ParticipantSeat.tsx:34`).
- **`StoredParticipant` vs. `Participant`:** Der Server speichert die echte Stimme und `lastSeen`, der Client bekommt nur, was er sehen darf. Zwei Types machen sichtbar, dass hier etwas verborgen wird.
- **`body: unknown` in `validation.ts`:** Von aussen kann alles kommen. Mit `unknown` erlaubt TypeScript keinen Zugriff, bevor wir mit `typeof` und `in` geprüft haben. Mit `any` würde TypeScript gar nichts prüfen; `any` ist bei uns per ESLint verboten.
- **Gemeinsame Types in `shared`:** Ändert jemand `SessionState`, melden Client und Server sofort Fehler, statt dass sie still aneinander vorbeireden.

B.8 «Welches Problem löst Context? Wann sind Props besser?»

> «Den Namen brauchen die Home-Seite und die Session-Seite. Die beiden sind Geschwister unter dem Router. Ohne Context müsste `App` den Namen halten und an jede Route geben (‹Prop Drilling›). Mit Context legt `main.tsx` den `UserProvider` einmal um alles, und jede Komponente liest den Namen mit `useContext(UserContext)`. Props sind besser für alles andere: Sie zeigen, woher die Daten kommen, und bei häufigen Änderungen wie dem Tisch-Zustand alle 1.5 s rendert nur der betroffene Teil neu. Context lohnt sich für wenige Werte, die selten wechseln und die viele Stellen brauchen.»

B.9 «Warum braucht es ein Cleanup im `useEffect`?»

> «Ein Effekt startet etwas, das ausserhalb von React weiterläuft: ein Intervall, einen Timer, eine Anmeldung am Server. Wird die Komponente entfernt, muss das wieder aufhören. Bei uns gibt es vier Cleanups:
>
> 1. **Polling der Liste** (`HomePage.tsx:60-63`): `clearInterval`, sonst liefen nach dem Wechsel zum Tisch weiter Requests für die Liste.
> 2. **Polling am Tisch** (`PokerRoom.tsx:72-75`): `clearInterval`, sonst liefe das Polling nach dem Verlassen weiter.
> 3. **Beitreten** (`SessionPage.tsx:81-92`): Leave-Request, sonst sässe man nach dem Verlassen noch bis zum Timeout am Tisch.
> 4. **«Kopiert!»** (`ShareLinkButton.tsx:15`): `clearTimeout`.

> Dazu kommt das `ignore`-Flag: Eine Antwort, die erst nach dem Cleanup ankommt, schreibt nicht mehr in den State.»

B.10 «Was macht StrictMode mit Effects?»

> «StrictMode (`main.tsx:17`) startet in der Entwicklung jeden Effekt beim ersten Anzeigen zweimal: Effekt, Cleanup, Effekt. So fällt sofort auf, wenn ein Cleanup fehlt. Beim Beitreten gehen dadurch zwei Join-Requests los. Das Cleanup des ersten Durchlaufs setzt dessen `ignore` auf `true`. Kommt seine Antwort, sieht der Code in `SessionPage.tsx:55-63`: ‹ich bin schon aufgeräumt›. Hat dieser Join einen neuen Teilnehmer angelegt, meldet er ihn sofort wieder ab. Übrig bleibt der Teilnehmer des zweiten Durchlaufs. Im gebauten Produktivcode läuft der Effekt nur einmal. Ein Test prüft das unter `<StrictMode>`.»

B.11 «Was passiert bei einem Reload am Tisch?»

> «Bei einem Reload läuft kein React-Cleanup, auf dem Server sitzt man also noch am Tisch. Nach dem Beitreten hat `SessionPage` die eigene `participantId` in `sessionStorage` geschrieben (Zeile 69). Die neu geladene Seite liest sie in Zeile 44 und schickt sie beim Beitreten mit. Der Server sieht in `sessions.ts`, Zeilen 98-106: ‹diese Person kenne ich noch› und antwortet 200 mit derselben ID, statt eine neue Person anzulegen. Platz und Stimme bleiben, es gibt keinen Doppel-Eintrag, und die Runde wird nicht aufgedeckt. Kennt der Server die ID nicht mehr (z. B. nach dem Timeout), legt er eine neue Person an (201). `sessionStorage` statt `localStorage`, weil jeder Tab eine eigene Person sein soll.»

B.12 «Warum habt ihr das so einfach gelöst? Kennt ihr keine Custom Hooks oder `AbortController`?»

Ehrlich antworten:

> «Die erste Fassung hatte einen allgemeinen Polling-Hook mit `useCallback`, `useRef` und `AbortController`. Wir haben den Code bewusst vereinfacht, damit jede Person jede Stelle erklären kann. Jetzt hat jede Seite, die pollt, ihren eigenen kurzen `useEffect` mit `setInterval` und Cleanup. Das ist zweimal fast derselbe Code, dafür sieht man alles an einer Stelle. Statt Requests abzubrechen, ignorieren wir späte Antworten mit dem `ignore`-Flag aus der React-Doku. Der Preis: Kommt eine alte Poll-Antwort zufällig nach der Antwort einer eigenen Aktion an, zeigt der Tisch höchstens 1.5 s lang den alten Stand, bis der nächste Poll kommt. Mit einem Custom Hook würden wir das Muster an einer Stelle sammeln, zum Beispiel `usePolling(load, intervall)`.»

Teil C – Fragenkatalog

#	Frage	Kurze Antwort
1	Was ist der Unterschied zwischen Props und State?	Props kommen von aussen und sind in der Komponente nur lesbar. State gehört der Komponente, ändert sich mit der Setter-Funktion und löst ein neues Rendern aus.
2	Warum darf man State nicht direkt ändern?	React merkt Änderungen nur über den Setter mit einem **neuen** Wert. Direkt verändert, rendert nichts neu, und andere Stellen sehen einen veränderten Wert. Darum `[...sessions]` vor `sort()` und `filter()` beim Löschen.

#	Frage	Kurze Antwort
3	Wozu dient `key` in Listen?	Damit React beim Ändern der Liste weiss, welches Element welches ist. Wir nehmen stabile IDs (`session.id`, `participant.id`), nie den Index.
4	Was macht `useEffect`, und wann läuft er?	Er führt Code **nach** dem Rendern aus, für alles ausserhalb von React (Server, Timer). Er läuft beim ersten Anzeigen und immer dann, wenn sich ein Wert im Dependency-Array ändert. `[]` = nur einmal.
5	Warum ist die Funktion im Effekt nicht direkt `async`?	Ein Effekt darf nur eine Cleanup-Funktion zurückgeben, eine `async`-Funktion gibt aber ein Promise zurück. Darum eine innere `async function` und sie sofort aufrufen.
6	Was macht das `ignore`-Flag?	Jeder Effekt-Durchlauf hat sein eigenes `ignore`. Das Cleanup setzt es auf `true`. Eine Antwort, die danach kommt, wird nicht mehr in den State geschrieben.
7	Warum ruft `loadSession` selbst `clearInterval` auf?	Wurde die Session gelöscht, ist weiteres Abfragen sinnlos. Sonst ginge alle 1.5 s ein Request mit 404 raus.
8	Was heisst `useState(() => …)`?	Die Funktion läuft nur beim ersten Render (Lazy Initial State). So wird `localStorage` nicht bei jedem Render gelesen.
9	Was heisst `setSessions((current) => …)`?	Der Setter bekommt eine Funktion und ruft sie mit dem neusten State auf. Sicher, auch wenn sich der State inzwischen geändert hat (z. B. ein Poll ist angekommen).
10	`localStorage` vs. `sessionStorage`?	`localStorage` bleibt nach dem Schliessen und gilt für alle Tabs → Name. `sessionStorage` gilt nur für einen Tab und übersteht einen Reload → `participantId`.
11	Warum Polling und nicht WebSockets?	Einfacher, passt zu REST, gehört zum Kursstoff. Nachteil: bis 1.5 s Verzögerung und mehr Requests. Für ein Team von 5–10 Personen kein Problem.
12	Warum 1.5 s am Tisch und 3 s bei der Liste?	Am Tisch soll es sich live anfühlen und der Heartbeat braucht Abstand zum 15-s-Timeout. Die Liste ist weniger dringend.
13	Was ist CORS und warum braucht ihr es nicht?	Der Browser blockiert Antworten von einer anderen Adresse (auch ein anderer Port zählt), wenn der Server es nicht erlaubt. Der Vite-Proxy macht `/api` zur selben Adresse wie die Seite.
14	Was macht `express.json()`?	Middleware, die JSON-Bodies liest und in `req.body` ablegt. Bei kaputtem JSON wirft sie einen `SyntaxError` → unser Error-Handler → 400.
15	Was ist Middleware?	Eine Funktion `(req, res, next)`, die vor oder nach den Routen läuft. Sie kann die Anfrage verändern, beantworten oder mit `next()` weitergeben.
16	Woran erkennt Express den Error-Handler?	An den vier Parametern `(err, req, res, next)`. Darum steht `_next` da, obwohl er nicht benutzt wird.
17	Warum ist `app.ts` von `index.ts` getrennt?	Damit Tests die App ohne `listen()` und ohne Timer importieren können.
18	Warum prüft der Server den Namen, obwohl der Client schon prüft?	Jeder kann mit `curl` Requests schicken. Prüfung im Client = Komfort, Prüfung im Server = Sicherheit. Dieselbe Funktion `validateName` sorgt für dieselbe Regel.
19	Warum 201 beim Erstellen und 200 beim Fortsetzen nach einem Reload?	201 Created heisst: etwas Neues ist entstanden. Beim Fortsetzen entsteht nichts Neues, darum 200 OK.
20	Warum 204 beim Löschen?	Erfolg ohne Inhalt, es gibt nichts zurückzuschicken.
21	Wann kommt 409?	Wenn jemand nach dem Aufdecken noch abstimmen will. Die Anfrage ist richtig gebaut, passt aber nicht zum aktuellen Zustand.

#	Frage	Kurze Antwort
22	Wie verhindert ihr, dass man Stimmen in den Entwicklertools sieht?	Der Server schickt vor dem Aufdecken nur `hasVoted`, `vote` ist `null` (`store.ts:38`). Ein E2E-Test prüft die echte Server-Antwort.
23	Warum sieht man die eigene Stimme vor dem Aufdecken?	Damit der Client die gewählte Karte aus dem Server-Zustand hervorheben kann, ohne sie selbst zu speichern. Für die eigene Person ist sie kein Geheimnis.
24	Wann wird automatisch aufgedeckt?	Wenn alle (mindestens eine Person) gewählt haben: nach einer Stimme oder wenn die letzte Person ohne Stimme geht.
25	Warum startet ein Deck-Wechsel eine neue Runde?	Alte Karten (z. B. `13`) gibt es im neuen Deck nicht, der Durchschnitt wäre sinnlos.
26	Wie zählen `?` und `☕`?	Nicht zum Durchschnitt, aber zur Anzahl Stimmen. Nur `?`/`☕` → Durchschnitt «–».
27	Was heisst Konsens bei euch?	Mindestens zwei Stimmen und alle gleich (auch zweimal `?`).
28	Welche Funktionen haben keine Nebenwirkungen?	`computeResult`, `validateName`, `toSessionState`: gleiche Eingabe, gleiches Ergebnis, sie ändern nichts. Darum leicht zu testen.
29	Unterschied `unknown` und `any`?	Beide nehmen alles an. Bei `unknown` verlangt TypeScript eine Prüfung vor der Nutzung, `any` schaltet die Prüfung ab. ESLint verbietet `any`.
30	Was macht `as JoinSessionRequest`?	Es sagt TypeScript, welche Form ein Wert hat. Es prüft nichts! Darum steht es erst **nach** der Prüfung in `validation.ts`. In `api.ts` vertrauen wir so dem eigenen Server.
31	Was bedeutet das `!` in `getElementById('root')!`?	«Dieser Wert ist sicher nicht `null`». Das `<div id="root">` steht in `index.html`.
32	Was machen `?.` und `??`?	`??` nimmt den rechten Wert, wenn links `null` oder `undefined` steht (`localStorage.getItem(…) ?? ''`). `?.` greift nur zu, wenn der Wert nicht `null`/`undefined` ist (im Test).
33	Was macht `async`/`await`?	`await` wartet auf ein Promise, ohne den Browser zu blockieren. Fehler fängt man mit `try`/`catch`.
34	Warum wirft `fetch` bei 404 keinen Fehler?	`fetch` wirft nur, wenn keine Antwort kam. Jede HTTP-Antwort ist für `fetch` ein Erfolg. Darum prüfen wir `response.ok` bzw. `response.status` selbst.
35	Warum gibt `getSession` bei 404 `null` zurück statt eines Fehlers?	«Gibt es nicht» ist ein normaler Fall, kein Absturz. `null` lässt sich mit einem `if` prüfen, ohne `try`/`catch`.
36	Warum `.catch(() => {})` beim Leave-Request?	Ein Fehler beim Verlassen ist egal, der Heartbeat-Timeout entfernt die Person trotzdem. Ohne `.catch` gäbe es eine «Uncaught (in promise)»-Meldung.
37	Unterschied `<Link>` und `<a href>`?	`<Link>` wechselt die Seite ohne Neuladen, der State bleibt. `<a href>` lädt alles neu. In der ErrorBoundary wollen wir genau das, darum `<Link reloadDocument>`.
38	Was ist ein kontrolliertes Formular?	Der Wert des Eingabefelds kommt aus dem State (`value`), jede Eingabe geht über `onChange` zurück. React ist die einzige Quelle für den Inhalt.
39	Warum `event.preventDefault()` im Formular?	Sonst lädt der Browser beim Absenden (Enter) die Seite neu.
40	Was macht `children` bei `PokerTable`?	Alles, was zwischen `<PokerTable>` und `</PokerTable>` steht, kommt als Prop `children` an. `PokerTable` zeigt es in der Mitte, ohne zu wissen, was es ist.
41	Wozu `key={participantId}` auf `PokerRoom`?	Neue ID → React baut ein neues `PokerRoom` mit leerem State, statt das alte weiterzuverwenden.

#	Frage	Kurze Antwort
42	Warum ist `ErrorBoundary` eine Klasse?	React bietet Error Boundaries nur als Klasse an (`getDerivedStateFromError`). Es ist die einzige Klasse der App.
43	Was macht `box-sizing: border-box`?	`width` schliesst Padding und Rahmen ein. Elemente werden nicht unerwartet breiter.
44	Flexbox oder Grid – wann was?	Flexbox für eine Richtung (Reihe von Buttons, Karten, Plätzen), Grid für Spalten und Zeilen (Home-Seite).
45	Was heisst Mobile First?	Zuerst Regeln für kleine Bildschirme schreiben, mit `min-width`-Media-Queries für grössere ergänzen.
46	Wozu `aria-pressed`, `role="alert"`, `role="status"`?	Screenreader erfahren, ob ein Button aktiv ist, und lesen Fehler sofort bzw. Ladehinweise höflich vor.
47	Was passiert, wenn zwei Personen gleichzeitig das Deck wechseln?	Der Server bearbeitet Requests nacheinander. Der zweite gewinnt, beide sehen beim nächsten Poll denselben Zustand.
48	Was passiert mit den Daten bei einem Neustart des Servers?	Alles weg (`Map` im Arbeitsspeicher). Bewusst so, es gibt keine Datenbank.
49	Wie testet ihr den Heartbeat, ohne 15 s zu warten?	`vi.useFakeTimers({ toFake: ['Date'] })` und `vi.advanceTimersByTime()` stellen die Uhr vor, dann wird `removeInactiveParticipants()` direkt aufgerufen.
50	Warum verlasst ihr die Session nicht beim Schliessen des Tabs (`pagehide`)?	`pagehide` feuert auch beim Reload, und der Browser unterscheidet die Fälle nicht. Ein Reload hätte die Stimme gelöscht und vielleicht für alle aufgedeckt. Ein geschlossener Tab bleibt darum bis 20 s am Tisch.
51	Was passiert mit zwei Tabs derselben Person?	Jeder Tab hat sein eigenes `sessionStorage`, also eine eigene `participantId`: zwei Einträge am Tisch.
52	Was macht `pnpm --filter`?	Führt einen Befehl nur in einem Paket des Workspaces aus.
53	Was prüfen die E2E-Tests, was die Unit-Tests nicht können?	Das Zusammenspiel im echten Browser mit echtem Server: zwei Personen gleichzeitig, Proxy, Reload, Tab schliessen, Layout ohne horizontales Scrollen, keine Fehler in der Konsole.
54	Was ersetzt der Fake-Server in den Client-Tests?	`fetch`. Er antwortet wie die echte API, damit die Tests ohne laufenden Server die Oberfläche prüfen können.

Teil D – Demo und Checkliste

D.1 Demo-Drehbuch (max. 3 Minuten, zwei Browserfenster)

Vorbereitung: Fenster **A** (normal, Name «Anna») links, Fenster **B** (privates Fenster, damit `localStorage` getrennt ist) rechts. Beide auf http://localhost:5173. Keine Session offen.

Zeit	Fenster	Aktion	Was ihr sagt
0:00	A	Name «Anna» steht schon da	«Der Name kommt aus dem `localStorage`, gelesen im `UserProvider`.»
0:10	B	Name leer lassen → Meldung, Buttons grau. Dann «Ben» eingeben	«`validateName` aus `shared` sperrt Erstellen und Beitreten. Der Server prüft mit derselben Funktion.»
0:25	A	«Neue Session erstellen»	«`POST /api/sessions` → 201, dann `navigate` auf `/sessions/<id>`. Die ID kommt per `useParams` in die Seite.»

Zeit	Fenster	Aktion	Was ihr sagt
0:40	B	Session erscheint in der Liste (≤ 3 s) → «Beitreten»	«Die Liste pollt alle 3 Sekunden.»
0:55	A	«Link kopieren» zeigen	«Für Teammitglieder, die direkt über den Link kommen.»
1:05	A	Karte **5** klicken	«`PUT …/vote`. Bei Ben erscheint nur eine verdeckte Karte, der Wert verlässt den Server nicht.»
1:20	B	Karte **8** klicken → automatisch aufgedeckt	«Alle haben gewählt → der Server deckt auf. Durchschnitt 6.5.»
1:35	A	«Neue Runde», beide wählen **3**	«Konsens wird hervorgehoben.»
1:55	B	Deck auf **T-Shirt**, A wählt S, B wählt L	«Ein Deck-Wechsel startet für alle eine neue Runde. S=2, L=5, Durchschnitt 3.5 → M.»
2:15	B	Seite neu laden (F5)	«Ben sitzt noch am Tisch, mit derselben Stimme: Die `participantId` liegt im `sessionStorage`.»
2:25	B	Tab schliessen	«Jetzt kommen keine Polls mehr. Nach höchstens 20 Sekunden entfernt der Server Ben (Heartbeat).»
2:35	A	Entwicklertools → Network: alle 1.5 s ein `GET …?participantId=`	«Das Polling ist zugleich der Heartbeat.»
2:50	A	«Verlassen» → Home	«Das Cleanup im `useEffect` von `SessionPage` schickt den DELETE.»

Plan B, falls etwas hängt: Seite neu laden. Ist der Server abgestürzt: Terminal zeigen, `pnpm dev` neu starten und dabei die Meldung «Server nicht erreichbar.» als Teil der Fehlerbehandlung erklären.

D.2 Checkliste für den Tag

Am Vortag

- [ ] Projekt aktuell, `pnpm install`
- [ ] `pnpm test` und `pnpm lint` grün (optional auch `pnpm test:e2e`)
- [ ] Demo einmal komplett mit Stoppuhr durchgespielt
- [ ] Jede Person kann Teil B ohne Unterlagen beantworten
- [ ] Laptop-Akku, Netzteil, ggf. Adapter für den Beamer

30 Minuten vorher

- [ ] Alle anderen Programme schliessen, Benachrichtigungen aus
- [ ] Terminal: `pnpm dev` läuft, `[server]` und `[client]` ohne Fehler
- [ ] Browser A und privater Browser B auf http://localhost:5173, Zoom 125 % für den Beamer
- [ ] Alte Test-Sessions löschen (oder Server neu starten)
- [ ] Editor offen mit diesen Dateien in Tabs: `shared/src/index.ts`, `server/src/routes/sessions.ts`, `server/src/store.ts`, `client/src/App.tsx`, `client/src/api.ts`, `client/src/UserContext.tsx`, `client/src/pages/HomePage.tsx`, `client/src/pages/SessionPage.tsx`, `client/src/components/PokerRoom.tsx`
- [ ] Zeilennummern im Editor eingeschaltet, Schrift gross genug

- ☐ Globale Suche (`Ctrl+Shift+F`), «Go to Line» (`Ctrl+G`) und «Go to Definition» (`F12`) geübt
- ☐ `DOKUMENTATION.pdf` offen als Rückfallebene

Während des Gesprächs

- Erst die Stelle im Code zeigen, dann erklären. Den deutschen Kommentar oben in der Datei vorlesen ist ein guter Einstieg.
- Ehrlich bleiben: «Das hat die KI so geschrieben, ich verstehe es so: …» ist besser als raten.
- Bei unbekannten Fragen laut denken und im Code nachsehen.

Teil E – Übungsaufgaben zum Selbsttest

Jede Aufgabe: Vermutung aufschreiben → ändern → `pnpm dev` bzw. `pnpm test` → beobachten → zurücksetzen mit `git checkout .` (ohne Git, z. B. aus dem ZIP: vorher eine Kopie des Ordners anlegen). Die Lösung steht darunter, erst nachher lesen.

1. Polling-Intervall – Setze `POLL_INTERVAL_MS` in `PokerRoom.tsx:19` auf `20_000`. Was passiert nach 20 s am Tisch?

> Der Server entfernt dich nach 15–20 s, weil der Heartbeat zu selten kommt. Beim nächsten Poll fehlst du in der Liste → «Du bist nicht mehr am Tisch». Zeigt: Polling und Timeout hängen zusammen.

2. Cleanup weglassen – Lösche in `HomePage.tsx:62` die Zeile `clearInterval(intervalId)`. Geh von der Home-Seite an einen Tisch und schau in den Network-Tab. Starte dann `pnpm test`.

> Die Liste wird weiter alle 3 s geladen, obwohl die Home-Seite gar nicht mehr zu sehen ist. Der Test «loads the list again every 3 seconds and stops polling after leaving the page» schlägt fehl.

3. StrictMode-Schutz entfernen – Lösche in `SessionPage.tsx:59-61` den Block, der den überflüssigen Teilnehmer wieder abmeldet. Tritt einer Session bei.

> Im Dev-Modus sitzt du zweimal am Tisch (einmal als Karteileiche, die nach 20 s verschwindet). Der Test «leaves the first participant again in StrictMode, so that only one remains» schlägt fehl.

4. Reload-Fortsetzung entfernen – Ersetze in `SessionPage.tsx:44` `sessionStorage.getItem(storageKey)` durch `null`. Wähle am Tisch eine Karte und lade die Seite neu (zweites Fenster offen lassen).

> Nach dem Reload sitzt du als neue Person am Tisch, deine Karte ist weg, und der alte Eintrag bleibt bis zu 20 s als Karteileiche sitzen. Der Test «continues as the same participant after a reload (sessionStorage)» schlägt fehl.

5. Kopie beim Sortieren – Ändere `HomePage.tsx:96` zu `sortedSessions = sessions`.

> Funktioniert scheinbar, `sort()` verändert jetzt aber den State direkt. Erkläre, warum das gefährlich ist (React bemerkt die Änderung nicht, der State ist nicht mehr der vom Server).

6. Geheimhaltung brechen – Ersetze in `store.ts:37-40` die Bedingung, sodass `vote` immer `participant.vote` ist.

> Vor dem Aufdecken stehen alle Stimmen in der Antwort (Network-Tab). Die Oberfläche zeigt sie trotzdem verdeckt. Das zeigt: Geheimhaltung muss auf dem Server passieren. Server-Tests schlagen fehl.

7. Automatisch aufdecken – Lösche in `store.ts:148-150` die Prüfung auf eine leere Session. Was passiert, wenn die letzte Person eine Session verlässt? Welcher Test schlägt fehl?

> Ohne Teilnehmer gibt es niemanden ohne Stimme, also wird die leere Session «aufgedeckt». Die nächste Person sieht einen aufgedeckten Tisch ohne Stimmen. Überraschung: **Kein** Test schlägt fehl. Schreibe einen supertest-Test, der das abdeckt (Session erstellen, beitreten, verlassen, `revealed` muss `false` sein).

8. Neue Karte – Füge `'40'` ins Fibonacci-Deck in `shared/src/index.ts:20` ein.

> Die Karte erscheint sofort im Client **und** der Server akzeptiert sie, weil beide `DECKS` aus `shared` lesen. Der Durchschnitt funktioniert ohne weitere Änderung.

9. Neuer Deck-Typ – Ergänze `DeckType` in `shared/src/index.ts:7` um `'powers'` (ohne `DECKS` anzupassen). Starte `pnpm typecheck`.

> TypeScript meldet Fehler überall, wo `DECKS[deck]` steht (z. B. `validation.ts:40`): Für `'powers'` gibt es in `DECKS` keinen Eintrag. So zeigt TypeScript alle Stellen, die man noch anpassen muss.

10. Statuscode – Ändere in `sessions.ts:139` den Code 409 zu 400. Welcher Test schlägt fehl, und warum ist 409 passender?

11. Fehler simulieren – Stoppe während einer Runde nur den Server (`Ctrl+C`, dann nur den Client starten mit `pnpm --filter @scrum-poker/client dev`).

> Meldung «Server nicht erreichbar. Neuer Versuch läuft …», der Tisch bleibt sichtbar. Server neu starten → Daten weg → «Diese Session existiert nicht (mehr)». Erkläre warum.

12. Context ohne Provider – Entferne `<UserProvider>` und `</UserProvider>` in `main.tsx`. Versuche, einen Namen einzugeben.

> Das Feld bleibt leer. Ohne Provider gilt der Startwert aus `UserContext.tsx:16`: Name `''` und ein `setName`, das nichts tut. Erkläre, warum der Provider über dem Router liegen muss.

13. `preventDefault` vergessen – Lösche `event.preventDefault()` in `NameForm.tsx:18` und drücke im Namensfeld Enter.

> Die Seite lädt neu (das Formular wird an den Browser geschickt). Der Name ist trotzdem da, weil er im `localStorage` liegt.

14. Breakpoint – Ändere in `HomePage.css:40` `768px` auf `1200px` und verkleinere das Fenster langsam. Wo springt das Layout um?

15. Box-Modell – Entferne `box-sizing: border-box` in `global.css:46`. Wo wird etwas breiter?

> Das Namensfeld und der Button «Neue Session erstellen» haben `width: 100%` plus Padding und Rahmen. Mit `content-box` werden sie breiter als ihre Karte und ragen über den Rand.

16. Neue Route – Füge eine Seite `/hilfe` hinzu (Komponente in `pages/`, Route in `App.tsx`, Link im Header). Erkläre jeden Schritt.

17. Neue API-Route – Baue `GET /api/sessions/:sessionId/result`, die `computeResult()` für eine aufgedeckte Session zurückgibt (sonst 409). Welche Methode, welche Statuscodes, welcher Type in `shared`? Schreibe einen supertest-Test.

18. Test lesen – Erkläre den Test «hides the votes until the cards are revealed» in `server/src/app.test.ts` Zeile für Zeile.

19. Datenfluss rückwärts – Starte bei `RoundResult.tsx` und verfolge, woher `participants` kommt, bis zum Eintrag in der `Map` im Server.

20. Zufällige Zeile – Eine Person öffnet eine beliebige Datei und zeigt auf eine beliebige Zeile. Die andere erklärt: Was passiert hier? Warum steht das hier? Was ginge kaputt, wenn man die Zeile löscht? So läuft das Fachgespräch ab.